Data Fetching

useEffect y buenas prácticas

📝 2026-01 — ⏱️ 60 min.

useEffect

Qué es useEffect?

Un hook para **sincronizar un componente con algo externo** al flujo de React.

Ejemplos de uso

- Llamadas a una API
- Suscripciones (WebSocket, eventos del DOM)
- Timers (`setInterval` , `setTimeout`)
- Leer o escribir en `localStorage`

Sintaxis de useEffect

```
useEffect(() => {
  // código que se ejecuta después del render
}, [dependencias]); // controla cuándo se ejecuta
```

Array de dependencias	Cuándo se ejecuta?
`[]` — vacío	Una vez al montar el componente
`[id, nombre]` — con variables	Cada vez que `id` o `nombre` cambien
Sin array	Después de **cada** render ⚠️

Sintaxis de useEffect

```
// Se ejecuta solo una vez (al montar)
useEffect(() => {
  console.log('Componente montado');
}, []);

// Se ejecuta cada vez que cambia `id`
useEffect(() => {
  console.log('El id cambió a:', id);
}, [id]);
```

El array de dependencias

✅

```
function Perfil({ id }) {
  const [usuario, setUsuario] = useState(null);

  useEffect(() => {
    fetch(`/api/usuario/${id}`)
      .then(r => r.json())
      .then(setUsuario);
  }, [id]); // Se ejecuta el useEffect siempre que `id` cambie

  return <p>{usuario?.nombre}</p>;
}
```

❌

```
function Perfil({ id }) {
  const [usuario, setUsuario] = useState(null);

  useEffect(() => {
    fetch(`/api/usuario/${id}`)
      .then(r => r.json())
      .then(setUsuario);
  }, []);

  // Si cambia id, el componente muestra
  // datos del usuario anterior
  return <p>{usuario?.nombre}</p>;
}
```

Cleanup en useEffect

Si el efecto inicia algo continuo (timer, suscripción, fetch), hay que **limpiarlo** cuando el componente se desmonta.

Retorna una función desde el efecto:

```
useEffect(() => {
  // Inicia el timer
  const intervalo = setInterval(() => {
    console.log('tick');
  }, 1000);

  // Cleanup: se ejecuta cuando el componente se desmonta
  return () => {
    clearInterval(intervalo);
  };
}, []);
```

> Sin cleanup: el timer sigue corriendo → **memory leak**.

Fetching de Datos

fetch() básico

`fetch` es una API nativa del navegador para hacer peticiones HTTP.

```
// Retorna una Promise con la respuesta
fetch('https://jsonplaceholder.typicode.com/posts')
  .then(respuesta => respuesta.json())   // convierte a JSON
  .then(datos => console.log(datos));    // usa los datos
```

Métodos HTTP más comunes:

```
// GET (leer) — por defecto
fetch('/api/posts')

// POST (crear)
fetch('/api/posts', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ titulo: 'Nuevo post' }),
})
```

fetch en un componente

```
function ListaDePosts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(datos => setPosts(datos));
  }, []);

  return (
    <ul>
      {posts.map(post => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  );
}
```

async/await con useEffect

No se puede hacer el callback de `useEffect` directamente `async`. En cambio, se define una función interna:

```
useEffect(() => {
  async function cargar() {
    const res = await fetch('https://jsonplaceholder.typicode.com/posts');
    const datos = await res.json();
    setPosts(datos);
  }

  cargar();
}, []);
```

> **Por qué?** useEffect espera una función de cleanup, no una Promise.

Loading y Error states

Un fetch puede estar en tres estados. Siempre hay que manejar los tres:

```
function ListaDePosts() {
  const [posts, setPosts] = useState([]);
  const [estado, setEstado] = useState({cargando: true, error: false});

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch('https://jsonplaceholder.typicode.com/posts');
        if (!res.ok) throw new Error('Error en la respuesta');
        const datos = await res.json();
        setPosts(datos);
        setEstado({cargando: false, error: false});
      } catch (err) {
        setEstado({cargando: false, error: true});
      }
    }
    cargar();
  }, []);

  if (estado.error) return <p>Error: {estado.error}</p>;
  if (estado.cargando) return <p>Cargando...</p>;
```

Buenas Prácticas

Buenas prácticas al hacer fetch

- ✅ **Siempre** manejar estados de `cargando` y `error`
- ✅ Usar `try/catch/finally` para capturar errores correctamente
- ✅ Verificar `res.ok` antes de parsear el JSON
- ✅ Cancelar el fetch si el componente se desmonta (**AbortController**)
- ✅ Extraer la lógica de fetch a un **custom hook** reutilizable
- ❌ No hacer fetch directamente en el cuerpo del componente (sin useEffect)
- ❌ No ignorar los errores con `.catch(() => {})`
- ❌ No actualizar el estado de un componente desmontado

Cancelar fetches con AbortController

```
useEffect(() => {
  const controller = new AbortController();

  async function cargar() {
    try {
      const res = await fetch('/api/datos', {
        signal: controller.signal, // vincula el fetch al controller
      });
      const datos = await res.json();
      setDatos(datos);
      setEstado({cargando: false, error: false});
    } catch (err) {
      if (err.name === 'AbortError') return; // fetch cancelado, ignorar
      setEstado({cargando: false, error: true});
    }
  }

  cargar();

  // Cleanup: cancela el fetch si el componente se desmonta
  return () => controller.abort();
}, []);
```

> Evita el error: "*Can't perform a React state update on an unmounted component*".

Custom Hook: useFetch

```
// useFetch.js
import { useState, useEffect } from 'react';

function useFetch(url) {
  const [datos, setDatos] = useState(null);
  const [estado, setEstado] = useState({cargando: true, error: false});

  useEffect(() => {
    async function cargar() {
      try {
        const res = await fetch(url);
        if (!res.ok) throw new Error('Error en el fetch');
        const json = await res.json();
        setDatos(json);
        setEstado({cargando: false, error: false});
      } catch (err) {
        if (err.name === 'AbortError') return; // fetch cancelado, ignorar
        setEstado({cargando: false, error: true});
      }
    }
    cargar();
  }, [url]);

  return { datos, ...estado };
}
```

Custom Hook: useFetch

```
// En cualquier componente
import useFetch from './useFetch';

function Posts() {
  const { datos, cargando, error } = useFetch(
    'https://jsonplaceholder.typicode.com/posts'
  );

  if (error) return <p>Error: {error}</p>;
  if (cargando) return <p>Cargando...</p>;

  return (
    <ul>
      {datos.map(p => (
        <li key={p.id}>{p.title}</li>
      ))}
    </ul>
  );
}
```

> Los custom hooks reutilizan lógica sin duplicar código.

TanStack Query (React Query)

Para proyectos reales, considera usar **TanStack Query**: maneja automáticamente loading, error, caché y re-fetching.

```
npm install @tanstack/react-query
```

```
import { useQuery } from '@tanstack/react-query';

function Posts() {
  const { data, isLoading, error } = useQuery({
    queryKey: ['posts'],
    queryFn: () =>
      fetch('https://jsonplaceholder.typicode.com/posts').then(r => r.json()),
  });

  if (isLoading) return <p>Cargando...</p>;
  if (error)     return <p>Error: {error.message}</p>;

  return <ul>{data.map(p => <li key={p.id}>{p.title}</li>)}</ul>;
}
```

Beneficios extra: caché automático, revalidación en foco, paginación, mutaciones.